```
// 文件：src/minigame/entry-douyin.ts
/**
 * 抖音小游戏入口（打包为 dist/douyin/crush-bundle.js，由 game.js shell require）。
 */
import './installTtShim';
import './douyinEarly';
import { launchDouyinGame } from './douyinLaunch';
void launchDouyinGame().catch((err: unknown) => {
  console.error('[crush-crush] douyin boot failed', err);
});
// 文件：src/minigame/installTtShim.ts
import { installTtWxShim } from './ttWxShim';
installTtWxShim();
// 文件：src/minigame/douyinEarly.ts
import { paintMiniGameStartupSplash } from './startupSplash';
console.info('[crush-crush] douyin entry loaded');
paintMiniGameStartupSplash();
// 文件：src/minigame/douyinLaunch.ts
import { Bootstrap, createAppContainer } from '../services/Bootstrap';
import { WxCanvasGameApp } from '../presentation/wechat/WxCanvasGameApp';
import { installTtWxShim } from './ttWxShim';
function waitHostBridge(): Promise<void> {
  return new Promise((resolve) => {
    let settled = false;
    const done = (): void => {
      if (settled) {
        return;
      }
      settled = true;
      resolve();
    };
    try {
      if (typeof wx !== 'undefined' && typeof wx.nextTick === 'function') {
        wx.nextTick(done);
      }
    } catch {
      // ignore
    }
    setTimeout(done, 48);
  });
}
export async function launchDouyinGame(): Promise<void> {
  if (!installTtWxShim()) {
    throw new Error('tt runtime missing');
  }
  await waitHostBridge();
  const container = createAppContainer();
  const bootstrap = new Bootstrap(container);
  const session = await bootstrap.run();
  const app = new WxCanvasGameApp(session);
```

```
  app.bootIntoLobby();
  app.start();
  await new Promise<void>((resolve) => {
    setTimeout(resolve, 50);
  });
  await app.preloadAssets();
}
// 文件：src/minigame/ttWxShim.ts
/**
 * 抖音小游戏运行时提供 `tt`，与微信 `wx` 大量 API 同名。
 * 在入口最前安装 shim，复用现有 Wx* 适配器与 Canvas 表现层。
 */
import { markMiniGameHost } from '../core/utils/miniGameHost';
type TtLike = typeof tt;
function fallbackMenuButtonRect(): TtMenuButtonRect {
  let windowWidth = 375;
  let statusBarHeight = 20;
  try {
    if (typeof tt !== 'undefined' && typeof tt.getSystemInfoSync === 'function') {
      const info = tt.getSystemInfoSync();
      windowWidth = info.windowWidth || info.screenWidth || windowWidth;
      statusBarHeight = info.statusBarHeight ?? statusBarHeight;
    }
  } catch {
    // ignore
  }
  const width = 87;
  const height = 32;
  const top = statusBarHeight + 4;
  const right = windowWidth - 7;
  return {
    width,
    height,
    top,
    right,
    bottom: top + height,
    left: right - width,
  };
}
function wrapCreateCustomAdFromBanner(ttApi: TtLike): void {
  if (typeof ttApi.createBannerAd !== 'function') {
    return;
  }
  const createBannerAd = ttApi.createBannerAd.bind(ttApi);
  (ttApi as unknown as { createCustomAd?: typeof wx.createCustomAd }).createCustomAd = (opts) => {
    const style = opts.style;
    return createBannerAd({
      adUnitId: opts.adUnitId,
      style: {
        left: style.left,
```

```
        top: style.top,
        width: style.width ?? 300,
        height: style.height,
      },
    });
  };
}
function patchReportEvent(ttApi: TtLike): void {
  if (typeof (ttApi as { reportEvent?: unknown }).reportEvent === 'function') {
    return;
  }
  if (typeof ttApi.reportAnalytics === 'function') {
    (ttApi as { reportEvent: typeof wx.reportEvent }).reportEvent = (event, data) => {
      ttApi.reportAnalytics?.(event, data ?? {});
    };
  }
}
function patchMenuButton(ttApi: TtLike): void {
  if (typeof ttApi.getMenuButtonBoundingClientRect === 'function') {
    return;
  }
  ttApi.getMenuButtonBoundingClientRect = fallbackMenuButtonRect;
}
function patchCreateImage(ttApi: TtLike): void {
  if (typeof ttApi.createImage === 'function') {
    return;
  }
  (ttApi as { createImage?: () => WxImage }).createImage = () => {
    const canvas = ttApi.createCanvas?.();
    if (canvas) {
      return canvas as unknown as WxImage;
    }
    throw new Error('tt.createImage unavailable');
  };
}
/** 让 readMiniProgramEnvVersion 能读到 develop / trial / release */
function patchAccountInfo(ttApi: TtLike): void {
  if (typeof ttApi.getAccountInfoSync === 'function') {
    return;
  }
  ttApi.getAccountInfoSync = () => {
    let envVersion: 'develop' | 'trial' | 'release' = 'release';
    try {
      const envType = (
        ttApi.getEnvInfoSync?.() as { microapp?: { envType?: string } } | undefined
      )?.microapp?.envType;
      if (envType === 'development' || envType === 'develop') {
        envVersion = 'develop';
      } else if (envType === 'preview' || envType === 'trial') {
        envVersion = 'trial';
```

```
      }
    } catch {
      // ignore
    }
    return { miniProgram: { envVersion } };
  };
}
function patchStorageFromSync(ttApi: TtLike): void {
  const getSync = ttApi.getStorageSync?.bind(ttApi);
  const setSync = ttApi.setStorageSync?.bind(ttApi);
  const removeSync = ttApi.removeStorageSync?.bind(ttApi);
  const clearSync = ttApi.clearStorageSync?.bind(ttApi);
  if (!getSync) {
    return;
  }
  if (typeof ttApi.getStorage !== 'function') {
    ttApi.getStorage = (opts) => {
      try {
        const data = getSync(opts.key);
        opts.success?.({ data, errMsg: 'getStorage:ok' });
      } catch {
        opts.fail?.({ errMsg: 'getStorage:fail data not found' });
      }
      opts.complete?.();
    };
  }
  if (setSync && typeof ttApi.setStorage !== 'function') {
    ttApi.setStorage = (opts) => {
      try {
        setSync(opts.key, opts.data);
        opts.success?.({ errMsg: 'setStorage:ok' });
      } catch (err) {
        opts.fail?.({ errMsg: String(err) });
      }
      opts.complete?.();
    };
  }
  if (removeSync && typeof ttApi.removeStorage !== 'function') {
    ttApi.removeStorage = (opts) => {
      try {
        removeSync(opts.key);
        opts.success?.({ errMsg: 'removeStorage:ok' });
      } catch {
        opts.fail?.({ errMsg: 'removeStorage:fail' });
      }
      opts.complete?.();
    };
  }
  if (clearSync && typeof ttApi.clearStorage !== 'function') {
    ttApi.clearStorage = (opts) => {
```

```
      try {
        clearSync();
        opts.success?.({ errMsg: 'clearStorage:ok' });
      } catch (err) {
        opts.fail?.({ errMsg: String(err) });
      }
      opts.complete?.();
    };
  }
}
function patchShowShareMenu(ttApi: TtLike): void {
  if (typeof ttApi.showShareMenu !== 'function') {
    return;
  }
  const raw = ttApi.showShareMenu.bind(ttApi);
  ttApi.showShareMenu = (opts?: { withShareTicket?: boolean; menus?: string[] }) => {
    try {
      raw({ withShareTicket: opts?.withShareTicket });
    } catch {
      raw();
    }
  };
}
/**
 * 将 tt 映射为全局 wx（仅当 wx 不存在时）。必须在其它模块读 wx 之前调用。
 */
export function installTtWxShim(): boolean {
  if (typeof tt === 'undefined') {
    return false;
  }
  const g = globalThis as { wx?: typeof wx; __CRUSH_HOST__?: string };
  if (typeof g.wx !== 'undefined') {
    return true;
  }
  patchMenuButton(tt);
  patchStorageFromSync(tt);
  patchCreateImage(tt);
  patchAccountInfo(tt);
  wrapCreateCustomAdFromBanner(tt);
  patchReportEvent(tt);
  patchShowShareMenu(tt);
  g.wx = tt as unknown as typeof wx;
  markMiniGameHost('douyin');
  return true;
}
// 文件：src/minigame/startupSplash.ts
import { bindMainCanvas2d, readWxSystemInfo } from '../core/utils/wxHost';
/** 重逻辑初始化前占住上屏 Canvas，避免 Helium 白屏。 */
export function paintMiniGameStartupSplash(): void {
  try {
```

```
    if (typeof wx === 'undefined') {
      return;
    }
    const binding = bindMainCanvas2d();
    const canvas = binding.canvas;
    const ctx = binding.ctx;
    const info = readWxSystemInfo();
    const cssW = Math.max(1, info.windowWidth || info.screenWidth || 375);
    const cssH = Math.max(1, info.windowHeight || info.screenHeight || 667);
    const dpr = Math.max(1, info.pixelRatio || 1);
    if (canvas.width < cssW * dpr * 0.5) {
      canvas.width = Math.floor(cssW * dpr);
      canvas.height = Math.floor(cssH * dpr);
    }
    ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
    ctx.fillStyle = '#b6e8fc';
    ctx.fillRect(0, 0, cssW, cssH);
    ctx.fillStyle = '#1864ab';
    ctx.font = 'bold 16px sans-serif';
    ctx.textAlign = 'center';
    ctx.textBaseline = 'middle';
    ctx.fillText('萌宠粉碎消 · 加载中…', cssW / 2, cssH * 0.48);
    binding.present?.();
  } catch (err) {
    console.warn('[crush-crush] startup splash failed', err);
  }
}
// 文件：src/config/buriedTileSprites.ts
/** 主线 1–20 关冰下埋藏：雪人 / 企鹅贴图（与 assets/main/tiles 一致） */
export const BURIED_TILE_SPRITE_SRC = {
  penguin: 'assets/main/tiles/penguin.png',
  snowman: 'assets/main/tiles/snowman.png',
} as const;
// 文件：src/core/adapters/HttpChallengeApi.ts
/**
 * 调本地或线上的好友挑战接口。微信里走 wx.request，单测和 Node 走 fetch。
 */
import challengeJson from '../../config/challenge.json';
import type {
  ChallengeStatusResponse,
  CreateChallengeRequest,
  CreateChallengeResponse,
  DailySubmitRequest,
  DailySubmitResponse,
  DailyTodayResponse,
  IChallengeApi,
  LeaderboardNotifyResponse,
  LeaderboardResponse,
  VerifyChallengeRequest,
  VerifyChallengeResponse,
```

```
} from '../ports/IChallengeApi';
async function requestJson(
  url: string,
  method: 'GET' | 'POST',
  body?: unknown,
): Promise<unknown> {
  if (typeof wx !== 'undefined' && typeof wx.request === 'function') {
    return new Promise((resolve, reject) => {
      wx.request({
        url,
        method,
        data: body,
        header: { 'content-type': 'application/json' },
        timeout: 1500,
        success: (res) => resolve(res.data),
        fail: (err) => reject(new Error(err.errMsg || 'request failed')),
      });
    });
  }
  const fetchFn = (globalThis as {
    fetch?: (
      input: string,
      init: { method: string; headers: Record<string, string>; body?: string },
    ) => Promise<{ json: () => Promise<unknown> }>;
  }).fetch;
  if (typeof fetchFn === 'function') {
    const res = await fetchFn(url, {
      method,
      headers: { 'content-type': 'application/json' },
      body: method === 'POST' ? JSON.stringify(body ?? {}) : undefined,
    });
    return res.json();
  }
  throw new Error('no http client');
}
export class HttpChallengeApi implements IChallengeApi {
  public constructor(private readonly baseUrl: string = challengeJson.baseUrl) {}
  public async create(body: CreateChallengeRequest): Promise<CreateChallengeResponse> {
    return this.post('/api/challenge/create', body) as Promise<CreateChallengeResponse>;
  }
  public async verify(body: VerifyChallengeRequest): Promise<VerifyChallengeResponse> {
    return this.post('/api/challenge/verify', body) as Promise<VerifyChallengeResponse>;
  }
  public async status(userId: string): Promise<ChallengeStatusResponse> {
    const url = `${this.base()}/api/challenge/status?user_id=${encodeURIComponent(userId)}&consume=
1`;
    return (await requestJson(url, 'GET')) as ChallengeStatusResponse;
  }
  public async dailyToday(userId: string): Promise<DailyTodayResponse> {
    const url = `${this.base()}/api/daily-challenge?user_id=${encodeURIComponent(userId)}`;
    if (!this.baseUrl) {
```

```
      return { ok: false, reason: 'no_api' };
    }
    return (await requestJson(url, 'GET')) as DailyTodayResponse;
  }
  public async dailySubmit(body: DailySubmitRequest): Promise<DailySubmitResponse> {
    return this.post('/api/daily-challenge/complete', body) as Promise<DailySubmitResponse>;
  }
  public async leaderboard(
    userId: string,
    maxClearedLevel = 0,
    profile?: { nickName?: string; avatarUrl?: string },
  ): Promise<LeaderboardResponse> {
    // scope=all：返回所有玩过本小游戏的用户；附带资料便于服务端入库展示
    const qs = [
      `user_id=${encodeURIComponent(userId)}`,
      `max_level=${Math.max(0, Math.floor(maxClearedLevel))}`,
      'scope=all',
    ];
    if (profile?.nickName) {
      qs.push(`nick_name=${encodeURIComponent(profile.nickName)}`);
    }
    if (profile?.avatarUrl) {
      qs.push(`avatar_url=${encodeURIComponent(profile.avatarUrl)}`);
    }
    const url = `${this.base()}/api/leaderboard?${qs.join('&')}`;
    if (!this.baseUrl) {
      return { ok: false, reason: 'no_api' };
    }
    return (await requestJson(url, 'GET')) as LeaderboardResponse;
  }
  public async leaderboardNotify(body: { userId: string; passerId?: string }): Promise<LeaderboardN
otifyResponse> {
    return this.post('/api/leaderboard/notify', body) as Promise<LeaderboardNotifyResponse>;
  }
  private base(): string {
    return this.baseUrl.replace(/\/$/, '');
  }
  private async post(path: string, body: unknown): Promise<unknown> {
    if (!this.baseUrl) {
      return { ok: false, reason: 'no_api' };
    }
    return requestJson(`${this.base()}${path}`, 'POST', body);
  }
}
// 文件：src/core/adapters/MahjongCdnFetch.ts
import type { MahjongThemeJsonFetcher } from '../../logic/mahjong/MahjongThemeCatalog';
/** 微信小游戏拉 CDN 上的麻将主题 manifest（JSON）。 */
export function createWxMahjongManifestFetcher(): MahjongThemeJsonFetcher {
  return (url: string) =>
    new Promise((resolve, reject) => {
      if (typeof wx === 'undefined' || typeof wx.request !== 'function') {
```

```
        reject(new Error('wx.request unavailable'));
        return;
      }
      wx.request({
        url,
        method: 'GET',
        success: (res) => {
          if (res.statusCode && res.statusCode >= 400) {
            reject(new Error(`HTTP ${res.statusCode}`));
            return;
          }
          resolve(res.data);
        },
        fail: (err) => reject(err),
      });
    });
}
// 文件：src/core/adapters/WxAdAdapter.ts
import type { AdPlacement, AdShowResult, IAdService } from '../ports/IAdService';
import { layoutLobbyBannerStyle, toNativeViewStyle } from '../utils/lobbyBanner';
import {
  canCreateWxFullscreenAds,
  canCreateWxNativeAds,
  isWxDevtoolsHost,
  readWxSystemInfo,
} from '../utils/wxHost';
export {
  isWxCustomAdHostSupported,
  canCreateWxNativeAds,
  canCreateWxFullscreenAds,
} from '../utils/wxHost';
/**
 * 广告位对应的微信广告单元 ID 配置。
 */
export interface WxAdUnitConfig {
  rewarded_revive: string;
  rewarded_crush_extend: string;
  interstitial_settle: string;
  banner_lobby?: string;
}
type RewardedPlacement = 'rewarded_revive' | 'rewarded_crush_extend';
type InterstitialPlacement = 'interstitial_settle';
interface RewardedSlot {
  ad: WxRewardedVideoAd;
  onLoad: () => void;
  onError: (err: { errMsg: string; errCode?: number }) => void;
}
interface InterstitialSlot {
  ad: WxInterstitialAd;
  onLoad: () => void;
```

```
  onError: (err: { errMsg: string; errCode?: number }) => void;
}
interface BannerSlot {
  ad: WxCustomAd;
  onLoad: () => void;
  onError: (err: { errMsg: string; errCode?: number }) => void;
  onHide: () => void;
  visible: boolean;
  /** show() 成功后才 hide/destroy，避免模拟器 removeImageView not found */
  attached: boolean;
}
function isRewardedPlacement(placement: AdPlacement): placement is RewardedPlacement {
  return placement === 'rewarded_revive' || placement === 'rewarded_crush_extend';
}
function isInterstitialPlacement(placement: AdPlacement): placement is InterstitialPlacement {
  return placement === 'interstitial_settle';
}
function isPlaceholderUnitId(adUnitId: string): boolean {
  return !adUnitId || adUnitId.startsWith('TODO_') || adUnitId.includes('UNIT_ID');
}
/**
 * 流量主「最大激励时长」15 秒：片子可能还是 30 秒，关广告时 isEnded 经常是 false。
 * 看够这段时间再关，按官方发奖处理。
 */
export const REWARDED_MIN_WATCH_MS = 14_000;
/** 插屏：show() 未成功前的等待；成功后改为等关闭，不再当 error。 */
export const INTERSTITIAL_WATCHDOG = {
  openMs: 8_000,
  watchMs: 180_000,
};
/** 激励：show() 仍在等待时不要判 not_ready；真正弹出后再等关闭。 */
export const REWARDED_WATCHDOG = {
  openMs: 12_000,
  pendingMaxMs: 60_000,
  watchMs: 180_000,
};
/**
 * 激励关闭是否算看完。
 * 缺 isEnded、看够 15 秒门槛、或明确播完，都发奖。
 */
export function isRewardedVideoCompleted(
  res?: { isEnded?: boolean } | null,
  shownMs = 0,
): boolean {
  if (shownMs >= REWARDED_MIN_WATCH_MS) {
    return true;
  }
  if (!res || typeof res.isEnded !== 'boolean') {
    return true;
  }
```

```
  return res.isEnded === true;
}
/**
 * 微信广告适配器：激励视频 / 插屏 / 大厅原生模板横幅；成对 off、展示后自动再预加载。
 * 仅本文件调用 wx 广告 API；逻辑层只依赖 IAdService。
 *
 * 说明：开发者工具不创建原生广告（insertTextView parent not found）。
 * 占位 adUnitId（TODO_*）时也跳过创建。
 */
export class WxAdAdapter implements IAdService {
  private readonly units: WxAdUnitConfig;
  private readonly readyMap = new Map<AdPlacement, boolean>();
  private readonly rewardedSlots = new Map<RewardedPlacement, RewardedSlot>();
  private readonly interstitialSlots = new Map<InterstitialPlacement, InterstitialSlot>();
  private bannerSlot: BannerSlot | null = null;
  /** 进关后禁止再露出；迟到的 show() 必须据此立刻 hide */
  private bannerWanted = false;
  private disposed = false;
  /**
   * @param units - 各广告位 adUnitId；可用 ad-placements.json 注入
   */
  public constructor(units: WxAdUnitConfig) {
    this.units = units;
  }
  /**
   * 预加载指定广告位。
   * @param placement - 广告位
   */
  public async load(placement: AdPlacement): Promise<void> {
    if (this.disposed) {
      return;
    }
    if (placement === 'banner_lobby') {
      const slot = this.ensureBanner();
      this.readyMap.set(placement, !!slot);
      return;
    }
    if (isRewardedPlacement(placement)) {
      const unitId =
        placement === 'rewarded_revive'
          ? this.units.rewarded_revive
          : this.units.rewarded_crush_extend;
      if (isPlaceholderUnitId(unitId)) {
        this.readyMap.set(placement, false);
        return;
      }
      const slot = this.ensureRewarded(placement);
      if (!slot) {
        this.readyMap.set(placement, false);
        return;
```

```
      }
      this.readyMap.set(placement, false);
      try {
        await slot.ad.load();
      } catch {
        this.readyMap.set(placement, false);
      }
      return;
    }
    if (isInterstitialPlacement(placement)) {
      if (isPlaceholderUnitId(this.units.interstitial_settle)) {
        this.readyMap.set(placement, false);
        return;
      }
      const existing = this.interstitialSlots.get(placement);
      const slot = this.ensureInterstitial(placement);
      if (!slot) {
        this.readyMap.set(placement, false);
        return;
      }
      if (!existing) {
        return;
      }
      try {
        await slot.ad.load();
      } catch (err) {
        console.warn('[crush-crush] interstitial load failed', err);
        this.readyMap.set(placement, false);
      }
    }
  }
  /**
   * 展示广告。
   * @param placement - 广告位
   * @returns completed / skipped / error / not_ready
   */
  public async show(placement: AdPlacement): Promise<AdShowResult> {
    if (this.disposed) {
      return 'error';
    }
    if (placement === 'banner_lobby') {
      return this.showBanner();
    }
    // 占位 ID：本地调试直接不可用，避免误触 SDK TextView 报错
    if (isRewardedPlacement(placement)) {
      const unitId =
        placement === 'rewarded_revive'
          ? this.units.rewarded_revive
          : this.units.rewarded_crush_extend;
      if (isPlaceholderUnitId(unitId)) {
```

```
        return 'not_ready';
      }
    }
    if (isInterstitialPlacement(placement) && isPlaceholderUnitId(this.units.interstitial_settle))
{
      return 'not_ready';
    }
    // 插屏创建后会自动拉取。未 onLoad 也要 show：否则进门广告永远 not_ready。
    if (isInterstitialPlacement(placement)) {
      return this.showInterstitial(placement);
    }
    if (!this.isReady(placement)) {
      await this.load(placement);
      if (!this.isReady(placement)) {
        return 'not_ready';
      }
    }
    if (isRewardedPlacement(placement)) {
      return this.showRewarded(placement);
    }
    return 'error';
  }
  /**
   * 广告是否就绪。
   * @param placement - 广告位
   */
  public isReady(placement: AdPlacement): boolean {
    return this.readyMap.get(placement) === true;
  }
  /**
   * 隐藏大厅横幅。激励 / 插屏无持续展示，忽略。
   * 即使本地还没记成可见，也要 hide：show() 可能仍在加载，进关后会迟到上屏。
   */
  public hide(placement: AdPlacement): void {
    if (placement !== 'banner_lobby' || this.disposed) {
      return;
    }
    this.bannerWanted = false;
    this.forceHideBanner();
  }
  private forceHideBanner(force = false): void {
    const slot = this.bannerSlot;
    if (!slot) {
      return;
    }
    const showing = force || slot.visible || slot.attached || slot.ad.isShow?.() === true;
    slot.visible = false;
    if (!showing) {
      return;
    }
    try {
```

```
      void slot.ad.hide();
    } catch {
      // 模拟器里视图没插上就会 removeImageView not found
    }
    slot.attached = false;
  }
  /**
   * 销毁全部广告实例并卸载监听，防止微信环境泄漏。
   */
  public dispose(): void {
    this.disposed = true;
    for (const slot of this.rewardedSlots.values()) {
      slot.ad.offLoad(slot.onLoad);
      slot.ad.offError(slot.onError);
      slot.ad.offClose();
      try {
        slot.ad.destroy();
      } catch {
        // ignore — destroy 时也可能触发 TextView not found
      }
    }
    this.rewardedSlots.clear();
    for (const slot of this.interstitialSlots.values()) {
      slot.ad.offLoad(slot.onLoad);
      slot.ad.offError(slot.onError);
      slot.ad.offClose();
      try {
        slot.ad.destroy();
      } catch {
        // ignore
      }
    }
    this.interstitialSlots.clear();
    this.destroyBanner();
    this.readyMap.clear();
  }
  private ensureRewarded(placement: RewardedPlacement): RewardedSlot | null {
    const existing = this.rewardedSlots.get(placement);
    if (existing) {
      return existing;
    }
    const adUnitId =
      placement === 'rewarded_revive'
        ? this.units.rewarded_revive
        : this.units.rewarded_crush_extend;
    if (isPlaceholderUnitId(adUnitId)) {
      return null;
    }
    if (!canCreateWxFullscreenAds()) {
      return null;
```

```
    }
    let ad: WxRewardedVideoAd;
    try {
      ad = wx.createRewardedVideoAd({ adUnitId });
    } catch (err) {
      console.warn('[crush-crush] createRewardedVideoAd failed', err);
      return null;
    }
    const onLoad = (): void => {
      this.readyMap.set(placement, true);
    };
    const onError = (): void => {
      this.readyMap.set(placement, false);
    };
    ad.onLoad(onLoad);
    ad.onError(onError);
    const slot: RewardedSlot = { ad, onLoad, onError };
    this.rewardedSlots.set(placement, slot);
    return slot;
  }
  private ensureInterstitial(placement: InterstitialPlacement): InterstitialSlot | null {
    const existing = this.interstitialSlots.get(placement);
    if (existing) {
      return existing;
    }
    if (isPlaceholderUnitId(this.units.interstitial_settle)) {
      return null;
    }
    if (!canCreateWxFullscreenAds()) {
      return null;
    }
    let ad: WxInterstitialAd;
    try {
      ad = wx.createInterstitialAd({
        adUnitId: this.units.interstitial_settle,
      });
    } catch (err) {
      console.warn('[crush-crush] createInterstitialAd failed', err);
      return null;
    }
    const onLoad = (): void => {
      this.readyMap.set(placement, true);
    };
    const onError = (err?: { errMsg: string; errCode?: number }): void => {
      console.warn('[crush-crush] interstitial error', err?.errCode, err?.errMsg);
      this.readyMap.set(placement, false);
    };
    ad.onLoad(onLoad);
    ad.onError(onError);
    const slot: InterstitialSlot = { ad, onLoad, onError };
```

```
    this.interstitialSlots.set(placement, slot);
    return slot;
  }
  private ensureBanner(): BannerSlot | null {
    if (this.bannerSlot) {
      return this.bannerSlot;
    }
    const adUnitId = this.units.banner_lobby ?? '';
    if (isPlaceholderUnitId(adUnitId)) {
      return null;
    }
    if (!canCreateWxNativeAds() || typeof wx.createCustomAd !== 'function') {
      return null;
    }
    const info = readWxSystemInfo();
    const safeBottom = Math.max(
      0,
      info.windowHeight - (info.safeArea?.bottom ?? info.windowHeight),
    );
    const box = toNativeViewStyle(
      layoutLobbyBannerStyle(
        info.windowWidth || info.screenWidth,
        info.windowHeight || info.screenHeight,
        safeBottom,
      ),
    );
    if (!box) {
      return null;
    }
    let ad: WxCustomAd;
    try {
      ad = wx.createCustomAd({
        adUnitId,
        adIntervals: 30,
        style: {
          left: box.left,
          top: box.top,
          width: box.width,
        },
      });
    } catch (err) {
      console.warn('[crush-crush] createCustomAd failed', err);
      return null;
    }
    const onLoad = (): void => {
      this.readyMap.set('banner_lobby', true);
      if (!this.bannerWanted) {
        this.forceHideBanner();
      }
    };
```

```
    const onError = (err?: { errMsg: string; errCode?: number }): void => {
      console.warn(
        '[crush-crush] lobby customAd error',
        err?.errCode,
        err?.errMsg,
      );
      this.readyMap.set('banner_lobby', false);
      if (this.bannerSlot) {
        this.bannerSlot.visible = false;
      }
    };
    const onHide = (): void => {
      if (this.bannerSlot) {
        this.bannerSlot.visible = false;
      }
    };
    ad.onLoad(onLoad);
    ad.onError(onError);
    ad.onHide?.(onHide);
    this.bannerSlot = { ad, onLoad, onError, onHide, visible: false, attached: false };
    return this.bannerSlot;
  }
  private destroyBanner(): void {
    const slot = this.bannerSlot;
    if (!slot) {
      return;
    }
    slot.ad.offLoad(slot.onLoad);
    slot.ad.offError(slot.onError);
    slot.ad.offHide?.(slot.onHide);
    if (slot.attached || !isWxDevtoolsHost()) {
      try {
        slot.ad.destroy();
      } catch {
        // ignore
      }
    }
    this.bannerSlot = null;
  }
  private async showBanner(): Promise<AdShowResult> {
    this.bannerWanted = true;
    const slot = this.ensureBanner();
    if (!slot) {
      this.readyMap.set('banner_lobby', false);
      return 'not_ready';
    }
    if (!this.bannerWanted) {
      this.forceHideBanner();
      return 'error';
    }
```

```
    if (slot.visible || slot.ad.isShow?.()) {
      if (!this.bannerWanted) {
        this.forceHideBanner();
        return 'error';
      }
      slot.visible = true;
      this.readyMap.set('banner_lobby', true);
      return 'completed';
    }
    try {
      await slot.ad.show();
      slot.attached = true;
      if (!this.bannerWanted) {
        this.forceHideBanner(true);
        return 'error';
      }
      slot.visible = true;
      this.readyMap.set('banner_lobby', true);
      return 'completed';
    } catch (err) {
      console.warn('[crush-crush] lobby customAd show failed', err);
      slot.visible = false;
      slot.attached = false;
      this.readyMap.set('banner_lobby', false);
      return 'error';
    }
  }
  private showRewarded(placement: RewardedPlacement): Promise<AdShowResult> {
    const slot = this.ensureRewarded(placement);
    if (!slot) {
      return Promise.resolve('not_ready');
    }
    this.readyMap.set(placement, false);
    return new Promise<AdShowResult>((resolve) => {
      let settled = false;
      let shown = false;
      let showInFlight = true;
      let watchdog = 0;
      const startedAt = Date.now();
      const unhookApp = (): void => {
        try {
          wx.offHide?.(onAppHide);
        } catch {
          // ignore
        }
        try {
          wx.offShow?.(onAppShow);
        } catch {
          // ignore
        }
```

```
      };
      const finish = (result: AdShowResult): void => {
        if (settled) {
          return;
        }
        settled = true;
        if (watchdog) {
          clearTimeout(watchdog);
          watchdog = 0;
        }
        unhookApp();
        slot.ad.offClose(onClose);
        resolve(result);
        void this.load(placement);
      };
      const shownMsNow = (): number => Math.max(0, Date.now() - startedAt);
      const onClose = (res?: { isEnded?: boolean }): void => {
        shown = true;
        finish(isRewardedVideoCompleted(res, shownMsNow()) ? 'completed' : 'skipped');
      };
      const onAppHide = (): void => {
        if (settled) {
          return;
        }
        shown = true;
        armWatchdog(180000);
      };
      const onAppShow = (): void => {
        if (settled || shownMsNow() < REWARDED_MIN_WATCH_MS) {
          return;
        }
        setTimeout(() => {
          if (settled) {
            return;
          }
          finish(isRewardedVideoCompleted({}, shownMsNow()) ? 'completed' : 'skipped');
        }, 400);
      };
      const armWatchdog = (ms: number): void => {
        if (settled) {
          return;
        }
        if (watchdog) {
          clearTimeout(watchdog);
        }
        watchdog = setTimeout(() => {
          if (settled) {
            return;
          }
          if (shown) {
```

```
            finish(
              isRewardedVideoCompleted({ isEnded: false }, shownMsNow())
                ? 'completed'
                : 'skipped',
            );
            return;
          }
          if (showInFlight && shownMsNow() < REWARDED_WATCHDOG.pendingMaxMs) {
            armWatchdog(REWARDED_WATCHDOG.openMs);
            return;
          }
          finish('not_ready');
        }, ms) as unknown as number;
      };
      slot.ad.onClose(onClose);
      try {
        wx.onHide?.(onAppHide);
        wx.onShow?.(onAppShow);
      } catch {
        // ignore
      }
      armWatchdog(REWARDED_WATCHDOG.openMs);
      slot.ad
        .show()
        .then(() => {
          showInFlight = false;
          shown = true;
          armWatchdog(REWARDED_WATCHDOG.watchMs);
        })
        .catch(() => {
          void slot.ad
            .load()
            .then(() => slot.ad.show())
            .then(() => {
              showInFlight = false;
              shown = true;
              armWatchdog(REWARDED_WATCHDOG.watchMs);
            })
            .catch(() => {
              showInFlight = false;
              if (shown) {
                armWatchdog(REWARDED_WATCHDOG.watchMs);
                return;
              }
              armWatchdog(600);
            });
        });
    });
  }
  private showInterstitial(placement: InterstitialPlacement): Promise<AdShowResult> {
```

```
    const slot = this.ensureInterstitial(placement);
    if (!slot) {
      return Promise.resolve('not_ready');
    }
    return new Promise<AdShowResult>((resolve) => {
      let settled = false;
      let shown = false;
      let watchdog = 0;
      const finish = (result: AdShowResult): void => {
        if (settled) {
          return;
        }
        settled = true;
        if (watchdog) {
          clearTimeout(watchdog);
          watchdog = 0;
        }
        slot.ad.offClose(onClose);
        resolve(result);
      };
      const onClose = (): void => {
        shown = true;
        finish('completed');
        void this.load(placement);
      };
      const armWatchdog = (ms: number): void => {
        if (settled) {
          return;
        }
        if (watchdog) {
          clearTimeout(watchdog);
        }
        watchdog = setTimeout(() => {
          if (settled) {
            return;
          }
          if (shown) {
            finish('completed');
            void this.load(placement);
            return;
          }
          finish('error');
        }, ms) as unknown as number;
      };
      armWatchdog(INTERSTITIAL_WATCHDOG.openMs);
      slot.ad.onClose(onClose);
      slot.ad
        .show()
        .then(() => {
          shown = true;
```

```
          armWatchdog(INTERSTITIAL_WATCHDOG.watchMs);
        })
        .catch((err: { errMsg?: string; errCode?: number }) => {
          console.warn(
            '[crush-crush] interstitial show failed',
            err?.errCode,
            err?.errMsg,
          );
          if (err?.errCode !== 2001) {
            void this.load(placement);
          }
          if (shown) {
            armWatchdog(INTERSTITIAL_WATCHDOG.watchMs);
            return;
          }
          finish('error');
        });
    });
  }
}
// 文件：src/core/adapters/WxAnalyticsAdapter.ts
import type { AnalyticsEventName, IAnalytics } from '../ports/IAnalytics';
/**
 * 微信埋点适配器骨架。
 * TODO: 对接 wx.reportEvent 或自建上报网关。
 */
export class WxAnalyticsAdapter implements IAnalytics {
  public track(event: AnalyticsEventName, props?: Record<string, string | number | boolean>): void
{
    if (typeof wx.reportEvent === 'function') {
      wx.reportEvent(event, props ?? {});
      return;
    }
    // 开发期兜底：控制台输出，避免逻辑层分支
    // eslint-disable-next-line no-console
    console.info('[analytics]', event, props ?? {});
  }
}
// 文件：src/core/adapters/WxAudioAdapter.ts
import type { AudioClipId, IAudio } from '../ports/IAudio';
import { isWxDesktopIdeHost } from '../utils/wxHost';
/** clipId → 本地资源路径（主包短音效 + BGM） */
const CLIP_SRC: Record<string, string> = {
  sfx_swap: 'assets/main/audio/sfx_swap.wav',
  sfx_match: 'assets/main/audio/sfx_match.wav',
  sfx_good: 'assets/main/audio/sfx_good.mp3',
  sfx_great: 'assets/main/audio/sfx_great.mp3',
  sfx_excellent: 'assets/main/audio/sfx_excellent.mp3',
  sfx_win: 'assets/main/audio/sfx_win.wav',
  sfx_fail: 'assets/main/audio/sfx_fail.wav',
  sfx_crush: 'assets/main/audio/sfx_crush.wav',
```

```
  sfx_ui: 'assets/main/audio/sfx_ui.wav',
  sfx_shuffle: 'assets/main/audio/sfx_shuffle.wav',
  sfx_hammer: 'assets/main/audio/sfx_hammer.wav',
  sfx_extra: 'assets/main/audio/sfx_extra.wav',
  bgm_main: 'assets/main/audio/bgm_main.mp3',
};
const BGM_ID = 'bgm_main';
const POOL_LIMIT = 8;
const SFX_CLIP_IDS = Object.keys(CLIP_SRC).filter((id) => id !== BGM_ID);
/**
 * 微信 InnerAudio 适配器：短音效池化 + 独立循环 BGM。
 *
 * 短音必须预加载，且禁止在 src 未就绪时 stop()+play()，
 * 否则延迟触发的 Good/Great/Excellent 会静默失败。
 */
export class WxAudioAdapter implements IAudio {
  private muted = false;
  private readonly pool = new Map<string, WxInnerAudioContext[]>();
  private readonly active = new Set<WxInnerAudioContext>();
  private readonly ready = new WeakSet<WxInnerAudioContext>();
  private readonly pendingPlay = new WeakSet<WxInnerAudioContext>();
  private bgm: WxInnerAudioContext | null = null;
  private bgmWanted = false;
  private bgmVolume = 0.38;
  private innerAudioOptionApplied = false;
  private bgmKeepAliveTimer = 0;
  private lastBgmPlayMs = 0;
  /** BGM 已成功 kick 过；paused 读不到时禁止被音效保活反复 play 造成爆音 */
  private bgmPlayingAssumed = false;
  private webAudioCtx: WxWebAudioContext | null | undefined;
  public play(clipId: AudioClipId, options?: { loop?: boolean; volume?: number }): void {
    if (clipId === BGM_ID || options?.loop) {
      this.playBgm(clipId, options?.volume);
      return;
    }
    if (this.muted) {
      return;
    }
    const src = CLIP_SRC[clipId];
    if (!src || typeof wx === 'undefined' || typeof wx.createInnerAudioContext !== 'function') {
      return;
    }
    this.applyInnerAudioOption();
    const ctx = this.acquire(src);
    ctx.loop = false;
    ctx.volume = options?.volume ?? 0.8;
    this.startSfx(ctx);
    // 仅在 BGM 明确被系统掐掉时再续；不要每次音效后都 play()，否则会叠音杂音。
    this.scheduleBgmKeepAlive();
  }
```

```
  public suspendForBackground(): void {
    this.bgmPlayingAssumed = false;
    if (!this.bgm) {
      return;
    }
    try {
      this.bgm.pause();
    } catch {
      try {
        this.bgm.stop();
      } catch {
        // ignore
      }
    }
  }
  public resumeFromBackground(): void {
    if (!isWxDesktopIdeHost()) {
      this.unlockWebAudio();
    }
    this.resumeBgm();
  }
  public preloadSfx(): void {
    if (typeof wx === 'undefined' || typeof wx.createInnerAudioContext !== 'function') {
      return;
    }
    this.applyInnerAudioOption();
    if (!isWxDesktopIdeHost()) {
      this.unlockWebAudio();
    }
    for (const id of SFX_CLIP_IDS) {
      const src = CLIP_SRC[id];
      if (src) {
        this.ensurePooled(src);
      }
    }
  }
  public stop(clipId: AudioClipId): void {
    if (clipId === BGM_ID) {
      this.stopBgm(false);
      return;
    }
    const src = CLIP_SRC[clipId];
    if (!src) {
      return;
    }
    const list = this.pool.get(src);
    if (!list) {
      return;
    }
    for (const ctx of list) {
```

```
      this.pendingPlay.delete(ctx);
      try {
        ctx.stop();
      } catch {
        // ignore
      }
    }
  }
  public stopAll(): void {
    for (const ctx of this.active) {
      this.pendingPlay.delete(ctx);
      try {
        ctx.stop();
      } catch {
        // ignore
      }
    }
    this.stopBgm(false);
  }
  public setMuted(muted: boolean): void {
    this.muted = muted;
    if (muted) {
      // 静音：停掉音效，暂停 BGM（保留 want 状态便于恢复）
      for (const ctx of this.active) {
        this.pendingPlay.delete(ctx);
        try {
          ctx.stop();
        } catch {
          // ignore
        }
      }
      if (this.bgm) {
        try {
          this.bgm.pause();
        } catch {
          try {
            this.bgm.stop();
          } catch {
            // ignore
          }
        }
        this.bgmPlayingAssumed = false;
      }
      return;
    }
    if (this.bgmWanted) {
      this.resumeBgm();
    }
  }
  public isMuted(): boolean {
```

```
    return this.muted;
  }
  public dispose(): void {
    if (this.bgmKeepAliveTimer) {
      clearTimeout(this.bgmKeepAliveTimer);
      this.bgmKeepAliveTimer = 0;
    }
    this.stopAll();
    if (this.bgm) {
      try {
        this.bgm.destroy();
      } catch {
        // ignore
      }
      this.bgm = null;
    }
    for (const list of this.pool.values()) {
      for (const ctx of list) {
        try {
          ctx.destroy();
        } catch {
          // ignore
        }
      }
    }
    this.pool.clear();
    this.active.clear();
  }
  private playBgm(clipId: AudioClipId, volume?: number): void {
    this.bgmWanted = true;
    if (typeof volume === 'number') {
      this.bgmVolume = volume;
    }
    if (this.muted) {
      return;
    }
    const src = CLIP_SRC[clipId] ?? CLIP_SRC[BGM_ID];
    if (!src || typeof wx === 'undefined' || typeof wx.createInnerAudioContext !== 'function') {
      return;
    }
    this.applyInnerAudioOption();
    if (!this.bgm) {
      this.bgm = this.createContext(false);
      this.bgm.src = src;
      this.bgm.loop = true;
      this.bgm.obeyMuteSwitch = false;
      this.bgm.onError((err) => {
        console.warn('[audio] bgm failed', src, err?.errMsg ?? err);
      });
    }
```

```
    this.bgm.volume = this.bgmVolume;
    if (!this.shouldKickBgm()) {
      return;
    }
    try {
      this.bgm.play();
      this.lastBgmPlayMs = Date.now();
      this.bgmPlayingAssumed = true;
    } catch {
      // ignore
    }
  }
  private scheduleBgmKeepAlive(): void {
    if (!this.bgmWanted || this.muted) {
      return;
    }
    // 已在播且 paused 可读为 false：不必保活。
    if (this.bgm && this.bgm.paused === false) {
      return;
    }
    // paused 读不到且我们认定仍在播：不要乱踢，避免爆音。
    if (this.bgm && this.bgm.paused !== true && this.bgmPlayingAssumed) {
      return;
    }
    if (this.bgmKeepAliveTimer) {
      clearTimeout(this.bgmKeepAliveTimer);
    }
    this.bgmKeepAliveTimer = setTimeout(() => {
      this.bgmKeepAliveTimer = 0;
      this.resumeBgm();
    }, 120) as unknown as number;
  }
  private resumeBgm(): void {
    if (!this.bgmWanted || this.muted) {
      return;
    }
    if (!this.bgm) {
      this.playBgm(BGM_ID, this.bgmVolume);
      return;
    }
    this.bgm.volume = this.bgmVolume;
    if (!this.shouldKickBgm()) {
      return;
    }
    try {
      this.bgm.play();
      this.lastBgmPlayMs = Date.now();
      this.bgmPlayingAssumed = true;
    } catch {
      // ignore
```

```
    }
  }
  /**
   * 仅在 BGM 确实需要重启时返回 true。
   * paused 读不到时禁止周期性 play()，否则微信里会叠轨产生沙沙声/爆音。
   */
  private shouldKickBgm(): boolean {
    if (!this.bgm) {
      return true;
    }
    if (this.bgm.paused === false) {
      this.bgmPlayingAssumed = true;
      return false;
    }
    if (this.bgm.paused === true) {
      this.bgmPlayingAssumed = false;
      return true;
    }
    return !this.bgmPlayingAssumed;
  }
  private stopBgm(clearWant: boolean): void {
    if (clearWant) {
      this.bgmWanted = false;
    }
    this.bgmPlayingAssumed = false;
    if (!this.bgm) {
      return;
    }
    try {
      this.bgm.stop();
    } catch {
      // ignore
    }
  }
  private applyInnerAudioOption(): void {
    if (this.innerAudioOptionApplied) {
      return;
    }
    this.innerAudioOptionApplied = true;
    if (typeof wx.setInnerAudioOption !== 'function') {
      return;
    }
    try {
      wx.setInnerAudioOption({
        obeyMuteSwitch: false,
        mixWithOther: true,
      });
    } catch {
      // 低版本基础库无此接口
    }
```

```
  }
  private unlockWebAudio(): void {
    try {
      if (this.webAudioCtx === undefined) {
        this.webAudioCtx = wx.createWebAudioContext?.() ?? null;
      }
      void this.webAudioCtx?.resume?.();
    } catch {
      // ignore
    }
  }
  private startSfx(ctx: WxInnerAudioContext): void {
    if (this.ready.has(ctx)) {
      this.pendingPlay.delete(ctx);
      try {
        ctx.seek?.(0);
      } catch {
        // ignore
      }
      try {
        ctx.play();
      } catch {
        // ignore
      }
      return;
    }
    this.pendingPlay.add(ctx);
    try {
      ctx.play();
    } catch {
      // src 未就绪时等 onCanplay 再播
    }
  }
  private ensurePooled(src: string): WxInnerAudioContext {
    let list = this.pool.get(src);
    if (!list) {
      list = [];
      this.pool.set(src, list);
    }
    if (list.length > 0) {
      return list[0]!;
    }
    const ctx = this.createSfxContext(src);
    list.push(ctx);
    return ctx;
  }
  private acquire(src: string): WxInnerAudioContext {
    let list = this.pool.get(src);
    if (!list) {
      list = [];
```

```
      this.pool.set(src, list);
    }
    for (const ctx of list) {
      if (!this.active.has(ctx)) {
        this.active.add(ctx);
        return ctx;
      }
    }
    if (list.length >= POOL_LIMIT) {
      const reuse = list[0]!;
      this.active.add(reuse);
      return reuse;
    }
    const ctx = this.createSfxContext(src);
    list.push(ctx);
    this.active.add(ctx);
    return ctx;
  }
  private createSfxContext(src: string): WxInnerAudioContext {
    const ctx = this.createContext(true);
    ctx.autoplay = false;
    ctx.loop = false;
    ctx.obeyMuteSwitch = false;
    ctx.onCanplay?.(() => {
      this.ready.add(ctx);
      if (this.pendingPlay.has(ctx)) {
        this.pendingPlay.delete(ctx);
        try {
          ctx.play();
        } catch {
          // ignore
        }
      }
    });
    ctx.onEnded(() => this.active.delete(ctx));
    ctx.onError((err) => {
      console.warn('[audio] sfx failed', src, err?.errMsg ?? err);
      this.active.delete(ctx);
      this.pendingPlay.delete(ctx);
    });
    ctx.src = src;
    return ctx;
  }
  private createContext(useWebAudio: boolean): WxInnerAudioContext {
    try {
      const useWeb = useWebAudio && !isWxDesktopIdeHost();
      return wx.createInnerAudioContext(
        useWeb ? { useWebAudioImplement: true } : undefined,
      );
    } catch {
```

```
      result,
    });
    return result === 'not_ready' ? 'unavailable' : 'error';
  }
  /** 大厅藤蔓：当前节点的 5 关（兼容旧逻辑） */
  public getVineNode(): VineNodeInfo {
    return getActiveVineNode(
      this.progress.getHighestLevelId(),
      this.getLevelCount(),
    );
  }
  /** 大厅地图：配置表里的每一段都展示，未解锁也能上滑看到。 */
  public listLobbyVineNodes(): VineNodeInfo[] {
    return listAllVineNodes(
      this.progress.getHighestLevelId(),
      this.getLevelCount(),
    );
  }
  /**
   * 仅供单测：覆盖当前总分（用于清洁门槛等断言）。
   */
  public forceScoreForTest(score: number): void {
    this.score = Math.max(0, Math.floor(score));
  }
  /**
   * 当前粉碎奖励会话（未开启时为 null）。
   */
  public getCrushSession(): CrushRewardModel | null {
    return this.crush;
  }
  /**
   * 粉碎加成分（未开启为 0）。
   */
  public getCrushScore(): number {
    return this.crush?.crushScore ?? 0;
  }
  /**
   * 本局粉碎点击次数。
   */
  public getCrushTapCount(): number {
    return this.crush?.tapCount ?? 0;
  }
  /**
   * 清洁小游戏门槛分。
   */
  public getCleanScoreThreshold(): number {
    return this.cleanScoreThreshold;
  }
  /**
   * 清洁小游戏时长（毫秒）。
```

```
   */
  public getCleanDurationMs(): number {
    return this.cleanDurationMs;
  }
  /**
   * 当前是否可询问「进入清洁模式」（通关结算且总分达标，本关仅一次）。
   */
  public canOfferCleaning(): boolean {
    return (
      this.fsm.getCurrent() === 'Settle' &&
      !this.cleaningUsedThisSettle &&
      this.score >= this.cleanScoreThreshold
    );
  }
  /**
   * 清洁模式会话。
   */
  public getCleanSession(): CrushRewardModel | null {
    return this.clean;
  }
  /**
   * 开始清洁小游戏：刷新盘面，限时点击清扫（不计主线胜负压力）。
   */
  public startCleaningMode(): boolean {
    if (!this.canOfferCleaning() || !this.board || !this.level) {
      return false;
    }
    if (!this.fsm.transitionTo('Cleaning')) {
      return false;
    }
    this.cleaningUsedThisSettle = true;
    const basicKinds = match3BasicKindCount(this.level.id, this.level);
    generatePlayableBoard(this.board, this.level.seed + 7777 + this.level.id, basicKinds);
    this.clean = new CrushRewardModel(this.cleanDurationMs, Math.max(1, this.crushTapPower));
    this.deps.analytics.track('crush_start', {
      levelId: this.level.id,
      mode: 'cleaning',
    });
    this.events.emit({ type: 'BoardChanged', version: this.board.version });
    return true;
  }
  /**
   * 结算时拒绝进入清洁模式（本关机会作废，不再弹窗）。
   */
  public declineCleaningOffer(): boolean {
    if (!this.canOfferCleaning()) {
      return false;
    }
    this.cleaningUsedThisSettle = true;
    return true;
```

```
  }
  /**
   * 结束清洁模式，回到结算。
   */
  public finishCleaningMode(): boolean {
    if (this.fsm.getCurrent() !== 'Cleaning') {
      return false;
    }
    this.deps.analytics.track('crush_end', {
      levelId: this.level?.id ?? 0,
      mode: 'cleaning',
      tapCount: this.clean?.tapCount ?? 0,
      cleanScore: this.clean?.crushScore ?? 0,
    });
    this.events.emit({
      type: 'CleaningEnded',
      cleanScore: this.clean?.crushScore ?? 0,
      tapCount: this.clean?.tapCount ?? 0,
    });
    this.clean = null;
    return this.fsm.transitionTo('Settle');
  }
  /**
   * 是否还有下一关配置。
   */
  public hasNextLevel(): boolean {
    if (!this.level) {
      return false;
    }
    return isMatch3LevelWithinCap(this.level.id + 1);
  }
  /**
   * 尝试交换两格并结算；结束后按目标与步数更新 FSM。
   */
  public trySwap(
    rowA: number,
    colA: number,
    rowB: number,
    colB: number,
  ): boolean {
    if (this.fsm.getCurrent() !== 'PlayerInput' || !this.board || !this.level || !this.goals) {
      return false;
    }
    if (!this.validator.canSwap(this.board, rowA, colA, rowB, colB)) {
      this.events.emit({ type: 'SwapRejected', rowA, colA, rowB, colB });
      return false;
    }
    this.fsm.transitionTo('Resolving');
    this.board.swap(rowA, colA, rowB, colB);
    this.movesLeft = Math.max(0, this.movesLeft - 1);
```

```
    this.playSfx('sfx_swap');
    this.events.emit({ type: 'SwapAccepted', rowA, colA, rowB, colB });
    const matches = this.finder.findMatches(this.board);
    const baseIndices: number[] = [];
    for (let i = 0; i < matches.size; i += 1) {
      baseIndices.push(matches.indices[i]!);
    }
    const kindA = this.board.getTile(rowA, colA);
    const kindB = this.board.getTile(rowB, colB);
    const sparkleCombo =
      this.board.isSparkle(rowA, colA) && this.board.isSparkle(rowB, colB);
    const specialMove =
      kindA === TileKind.ColorBomb ||
      kindB === TileKind.ColorBomb ||
      (isSpecialTile(kindA) && isSpecialTile(kindB)) ||
      sparkleCombo;
    const clearIndices = expandSpecialClears(
      this.board,
      rowA,
      colA,
      rowB,
      colB,
      baseIndices,
    );
    // 特殊交换但未产生任何清除（理论上不应发生）
    if (clearIndices.length === 0 && !specialMove) {
      this.board.swap(rowA, colA, rowB, colB);
      this.movesLeft += 1;
      this.fsm.transitionTo('PlayerInput');
      this.events.emit({ type: 'SwapRejected', rowA, colA, rowB, colB });
      return false;
    }
    if (clearIndices.length > 0) {
      this.playMatchPraiseSfx(Math.max(30, clearIndices.length * 10));
      this.firstWavePraisePlayed = true;
    }
    const firstKinds: TileKind[] = clearIndices.map(
      (index) => this.board!.cells[index] as TileKind,
    );
    const playback = this.capturePlaybackSnapshot();
    this.events.emit({
      type: 'TilesCleared',
      indices: clearIndices,
      kinds: firstKinds,
    });
    // 优先在玩家换入的那一格生成特殊块（开心消消乐手感）
    const preferredSpawnRow = rowB;
    const preferredSpawnCol = colB;
    const result = this.resolver.resolve(this.board, matches, {
      seed: this.level.seed + this.board.version,
```

```
      preferredSpawnRow,
      preferredSpawnCol,
      initialClearIndices: clearIndices,
      spawnSpecials: !(
        kindA === TileKind.ColorBomb || kindB === TileKind.ColorBomb
      ),
    });
    this.events.emit({
      type: 'ResolvePlayback',
      ...playback,
      waves: result.waves,
    });
    this.score += result.cleared * 10;
    this.goals.applyClearedKinds(result.clearedKinds);
    this.goals.syncScore(this.score);
    this.emitGoalProgress();
    if (result.fell.length > 0) {
      this.events.emit({
        type: 'TilesFell',
        moves: result.fell.map((f) => ({
          tileId: f.tileId,
          fromRow: f.fromRow,
          toRow: f.toRow,
          col: f.col,
        })),
      });
    }
    if (result.spawned.length > 0) {
      this.events.emit({
        type: 'TilesSpawned',
        indices: result.spawned.map((s) => this.board!.index(s.row, s.col)),
        tileIds: result.spawned.map((s) => s.tileId),
        kinds: result.spawned.map((s) => s.kind),
      });
    }
    this.events.emit({ type: 'CascadeDone', depth: result.cascadeDepth });
    this.events.emit({ type: 'BoardChanged', version: this.board.version });
    this.evaluateLevelOutcome();
    return true;
  }
  /**
   * 粉碎奖励 / 清洁模式：点击格子触发爆炸。
   * @returns 是否成功触发（状态/坐标非法时 false）
   */
  public tryCrushTap(row: number, col: number): boolean {
    const state = this.fsm.getCurrent();
    const burst =
      state === 'CrushReward' ? this.crush : state === 'Cleaning' ? this.clean : null;
    if (!this.board || !burst) {
      return false;
```

```
    }
    if (burst.isExpired()) {
      return false;
    }
    if (!this.board.inBounds(row, col)) {
      return false;
    }
    burst.tapCount += 1;
    const result = this.crushResolver.resolveTap(this.board, burst, row, col);
    const scoreAdded = result.clearedIndices.length * 10;
    burst.crushScore += scoreAdded;
    // 清洁模式不加主线总分，避免解压局扭曲闯关分数
    if (state === 'CrushReward') {
      this.score += scoreAdded;
    }
    const primary = result.fired[0];
    if (scoreAdded > 0) {
      this.playMatchPraiseSfx(scoreAdded);
    }
    this.events.emit({
      type: 'CrushBurstFired',
      row: primary?.epicenter.r ?? row,
      col: primary?.epicenter.c ?? col,
      radius: primary?.radius ?? burst.tapPower,
      clearedIndices: result.clearedIndices.slice(),
      scoreAdded,
    });
    this.events.emit({ type: 'BoardChanged', version: this.board.version });
    // 粉碎 / 清洁：盘面扫空后剩余立刻清零并弹出完成
    if (this.isBoardFullyCleared()) {
      burst.forceExpire();
      if (state === 'Cleaning') {
        this.finishCleaningMode();
      } else {
        this.finishCrushReward();
      }
    }
    return true;
  }
  /**
   * 推进粉碎/清洁倒计时；到期时自动收尾。
   * @returns 是否仍在进行中
   */
  public tickCrushReward(dtMs: number): boolean {
    const state = this.fsm.getCurrent();
    if (state === 'CrushReward' && this.crush) {
      if (this.isBoardFullyCleared()) {
        this.crush.forceExpire();
        this.finishCrushReward();
        return false;
```

```
      }
      const alive = this.crush.tick(dtMs);
      if (!alive) {
        this.finishCrushReward();
        return false;
      }
      return true;
    }
    if (state === 'Cleaning' && this.clean) {
      // 已扫空却仍有剩余：立刻清零收尾（防止空盘空等）
      if (this.isBoardFullyCleared()) {
        this.clean.forceExpire();
        this.finishCleaningMode();
        return false;
      }
      const alive = this.clean.tick(dtMs);
      if (!alive) {
        this.finishCleaningMode();
        return false;
      }
      return true;
    }
    return false;
  }
  /**
   * 观看激励视频延长粉碎时间。
   */
  public async watchAdToExtendCrush(): Promise<ReviveAdResult> {
    if (this.fsm.getCurrent() !== 'CrushReward' || !this.crush || !this.level) {
      return 'unavailable';
    }
    this.deps.analytics.track('ad_show', {
      placement: 'rewarded_crush_extend',
      levelId: this.level.id,
    });
    const result = await this.deps.ads.show('rewarded_crush_extend');
    if (result === 'completed') {
      const extra =
        typeof adPlacementsJson.rewarded_crush_extend.extraMs === 'number'
          ? adPlacementsJson.rewarded_crush_extend.extraMs
          : 5000;
      this.crush.extend(extra);
      this.deps.analytics.track('ad_complete', {
        placement: 'rewarded_crush_extend',
        extraMs: extra,
      });
      return 'revived';
    }
    if (result === 'skipped') {
      return 'skipped';
```

```
    }
    this.deps.analytics.track('ad_error', {
      placement: 'rewarded_crush_extend',
      result,
    });
    return result === 'not_ready' ? 'unavailable' : 'error';
  }
  /**
   * 粉碎结束：时间到或点领取。剩余小动物收尾爆炸加分，然后进结算，不必点完。
   * 注意：必须先进入 Settle，再发 CrushEnded，否则结算页询问清洁时状态仍是 CrushReward。
   */
  public finishCrushReward(): boolean {
    if (this.fsm.getCurrent() !== 'CrushReward') {
      return false;
    }
    let crushScore = 0;
    let leftoverCleared = 0;
    if (this.board && this.crush) {
      this.crush.clearQueue();
      const finale = this.crushResolver.resolveFinale(
        this.board,
        this.crush,
        this.crushFinaleMaxBursts,
      );
      leftoverCleared = finale.clearedIndices.length;
      const scoreAdded = leftoverCleared * 10;
      this.crush.crushScore += scoreAdded;
      this.score += scoreAdded;
      crushScore = this.crush.crushScore;
      if (scoreAdded > 0) {
        this.playMatchPraiseSfx(scoreAdded);
      }
      if (finale.fired.length > 0) {
        const primary = finale.fired[0]!;
        this.events.emit({
          type: 'CrushBurstFired',
          row: primary.epicenter.r,
          col: primary.epicenter.c,
          radius: primary.radius,
          clearedIndices: finale.clearedIndices.slice(),
          scoreAdded,
        });
        this.events.emit({ type: 'BoardChanged', version: this.board.version });
      }
      this.deps.analytics.track('crush_end', {
        levelId: this.level?.id ?? 0,
        crushScore: this.crush.crushScore,
        tapCount: this.crush.tapCount,
      });
      if (this.level) {
```

```
        this.commitLevelCleared();
      }
    }
    this.lastSettleWon = true;
    if (!this.fsm.transitionTo('Settle')) {
      return false;
    }
    this.events.emit({
      type: 'CrushEnded',
      crushScore,
      totalScore: this.score,
      leftoverCleared,
    });
    return true;
  }
  /**
   * 查询失败后的看广告复活报价（仅 LevelFailed 有效）。
   */
  public getReviveOffer(): ReviveOffer {
    if (this.fsm.getCurrent() !== 'LevelFailed' || !this.level) {
      return {
        allowed: false,
        movesGranted: 0,
        used: 0,
        maxPerLevel: 0,
      };
    }
    const decision = this.adPolicy.decideOnSettle(this.level.id, false);
    const offer = this.revivePolicy.offer();
    if (!decision.offerRewardedRevive) {
      return {
        allowed: false,
        movesGranted: 0,
        used: offer.used,
        maxPerLevel: offer.maxPerLevel,
      };
    }
    return offer;
  }
  /**
   * 观看激励视频复活：成功则加步并回到 PlayerInput。
   * @returns revived / skipped / unavailable / error
   */
  public async watchAdToRevive(): Promise<ReviveAdResult> {
    const offer = this.getReviveOffer();
    if (!offer.allowed || !this.level) {
      return 'unavailable';
    }
    this.deps.analytics.track('ad_show', {
      placement: 'rewarded_revive',
```

```
      levelId: this.level.id,
    });
    const result = await this.deps.ads.show('rewarded_revive');
    if (result === 'completed') {
      this.revivePolicy.consume();
      this.movesLeft += offer.movesGranted;
      this.deps.analytics.track('ad_complete', {
        placement: 'rewarded_revive',
        movesGranted: offer.movesGranted,
      });
      if (!this.fsm.transitionTo('PlayerInput')) {
        this.fsm.forceTo('PlayerInput');
      }
      return 'revived';
    }
    if (result === 'skipped') {
      return 'skipped';
    }
    this.deps.analytics.track('ad_error', {
      placement: 'rewarded_revive',
      result,
    });
    return result === 'not_ready' ? 'unavailable' : 'error';
  }
  /**
   * 通关离开结算时尝试插屏：点「下一关 / 回大厅」才弹，先让玩家看完胜利。
   * 未就绪或冷却中会静默跳过，不挡住去下一关。
   */
  public async maybeShowSettleInterstitial(): Promise<void> {
    if (this.fsm.getCurrent() !== 'Settle' || !this.level) {
      return;
    }
    const cooldownSeconds =
      typeof adPlacementsJson.interstitial_settle.cooldownSeconds === 'number'
        ? adPlacementsJson.interstitial_settle.cooldownSeconds
        : 60;
    const now = Date.now();
    if (
      !      this.adPolicy.shouldShowInterstitial(
        this.level.id,
        this.lastSettleWon,
        this.lastInterstitialMs,
        now,
        cooldownSeconds,
      )
    ) {
      return;
    }
    if (this.interstitialShowing) {
      return;
```

```
    }
    this.interstitialShowing = true;
    this.deps.analytics.track('ad_show', {
      placement: 'interstitial_settle',
      levelId: this.level.id,
    });
    try {
      const result = await this.deps.ads.show('interstitial_settle');
      if (result === 'completed' || result === 'skipped') {
        this.lastInterstitialMs = now;
      }
    } finally {
      this.interstitialShowing = false;
    }
  }
  /**
   * 进大厅插屏：广告一就绪就弹。微信启动后一小段时间会拦（2001），
   * 未弹出时返回 retry，由界面在大厅里接着试，直到真正弹出。
   */
  public async maybeShowLaunchInterstitial(): Promise<'shown' | 'retry' | 'stop'> {
    if (!this.isLaunchInterstitialEnabled()) {
      return 'stop';
    }
    if (this.launchInterstitialTried) {
      return 'stop';
    }
    if (this.fsm.getCurrent() !== 'Lobby') {
      return 'stop';
    }
    if (this.interstitialShowing) {
      return 'retry';
    }
    this.interstitialShowing = true;
    try {
      const result = await this.deps.ads.show('interstitial_settle');
      if (result === 'completed' || result === 'skipped') {
        this.launchInterstitialTried = true;
        this.lastInterstitialMs = Date.now();
        this.deps.analytics.track('ad_show', {
          placement: 'interstitial_settle',
          reason: 'launch',
        });
        return 'shown';
      }
      return 'retry';
    } finally {
      this.interstitialShowing = false;
    }
  }
  public isLaunchInterstitialEnabled(): boolean {
```

```
    return adPlacementsJson.interstitial_launch?.enabled === true;
  }
  /** 首页先画一帧再弹，毫秒。 */
  public getLaunchInterstitialDelayMs(): number {
    return this.readLaunchMs('delayMs', 800);
  }
  /** 被微信频控拦住后的重试间隔。 */
  public getLaunchInterstitialRetryMs(): number {
    return this.readLaunchMs('retryMs', 3000);
  }
  /** 大厅里最长等到何时放弃进门插屏。 */
  public getLaunchInterstitialTimeoutMs(): number {
    return this.readLaunchMs('timeoutMs', 35000);
  }
  private readLaunchMs(
    key: 'delayMs' | 'retryMs' | 'timeoutMs',
    fallback: number,
  ): number {
    const value = adPlacementsJson.interstitial_launch?.[key];
    return typeof value === 'number' && value >= 0 ? value : fallback;
  }
  /**
   * 大厅底部原生模板横幅：流量主已开通且填了广告位。
   */
  public isLobbyBannerEnabled(): boolean {
    const unit = adPlacementsJson.banner_lobby?.adUnitId;
    return (
      adPlacementsJson.banner_lobby?.enabled === true &&
      typeof unit === 'string' &&
      unit.startsWith('adunit-')
    );
  }
  /**
   * 大厅底部给横幅留的高度，设置/玩法按钮要抬上去。
   */
  public getLobbyBannerReservePx(): number {
    if (!this.isLobbyBannerEnabled()) {
      return 0;
    }
    const info = this.deps.platform.getSystemInfo();
    return lobbyBannerReserveHeight(
      info.windowWidth,
      info.windowHeight,
      info.safeAreaBottom ?? 0,
    );
  }
  /**
   * 在大厅展示底部横幅。进关、麻将馆对局或弹层时不要调。
   */
  public async showLobbyBanner(): Promise<AdShowResult> {
```

```
    if (!this.isLobbyBannerEnabled()) {
      this.hideLobbyBanner();
      return 'not_ready';
    }
    if (this.fsm.getCurrent() !== 'Lobby') {
      this.hideLobbyBanner();
      return 'error';
    }
    if (this.mahjong.getActiveGame()) {
      this.hideLobbyBanner();
      return 'error';
    }
    const result = await this.deps.ads.show('banner_lobby');
    if (this.fsm.getCurrent() !== 'Lobby' || this.mahjong.getActiveGame()) {
      this.hideLobbyBanner();
      return 'error';
    }
    if (result === 'completed') {
      this.deps.analytics.track('ad_show', { placement: 'banner_lobby' });
    }
    return result;
  }
  /**
   * 隐藏大厅横幅（进关、设置页、切后台）。
   */
  public hideLobbyBanner(): void {
    this.deps.ads.hide?.('banner_lobby');
  }
  /**
   * 画布起来后再预加载广告，避免启动瞬间创建原生广告卡死模拟器。
   */
  public warmupAds(): void {
    if (!canCreateWxFullscreenAds()) {
      return;
    }
    void this.deps.ads.load('rewarded_revive');
    void this.deps.ads.load('interstitial_settle');
    setTimeout(() => {
      void this.deps.ads.load('rewarded_crush_extend');
    }, 800);
  }
  /**
   * 失败确认后进入结算态（放弃复活）。
   */
  public acknowledgeFailure(): boolean {
    if (this.fsm.getCurrent() !== 'LevelFailed') {
      return false;
    }
    this.lastSettleWon = false;
    return this.fsm.transitionTo('Settle');
```

```
  }
  /**
   * 结算后进入下一关。
   * 插屏由界面在带遮罩的 runDuringAd 里先弹，这里只切关，避免连弹两次。
   * @returns 是否成功开局；无下一关时返回 false（保持结算态）
   */
  public async continueToNextLevel(): Promise<boolean> {
    if (this.fsm.getCurrent() !== 'Settle' || !this.level) {
      return false;
    }
    const nextId = this.level.id + 1;
    if (!isMatch3LevelWithinCap(nextId)) {
      return false;
    }
    await this.startLevel(nextId);
    return true;
  }
  /**
   * 结算后重试当前关。
   */
  public async retryLevel(): Promise<boolean> {
    if (this.fsm.getCurrent() !== 'Settle' || !this.level) {
      return false;
    }
    await this.startLevel(this.level.id);
    return true;
  }
  /**
   * 结算后返回大厅。
   */
  public returnToLobby(): boolean {
    if (this.fsm.getCurrent() !== 'Settle') {
      return false;
    }
    return this.fsm.transitionTo('Lobby');
  }
  /**
   * 任意对局态中途退出回首页（放弃当前关）。
   */
  public quitToLobby(): boolean {
    const state = this.fsm.getCurrent();
    if (state === 'Lobby' || state === 'Boot') {
      return true;
    }
    this.crush = null;
    this.clean = null;
    this.cleaningUsedThisSettle = false;
    this.board = null;
    this.goals = null;
    // level 保留最后一关信息也可清掉；回大厅不依赖它
```

```
    return this.fsm.forceTo('Lobby');
  }
  public dispose(): void {
    this.fsm.dispose();
    this.events.dispose();
    this.deps.ads.dispose();
    this.deps.audio.dispose();
    this.board = null;
    this.level = null;
    this.goals = null;
  }
  /**
   * 根据目标与步数判定胜负，并写入进度。
   */
  private evaluateLevelOutcome(): void {
    if (!this.level || !this.goals) {
      return;
    }
    if (this.goals.isAllCompleted()) {
      if (this.fsm.getCurrent() === 'PlayerInput') {
        this.fsm.transitionTo('Resolving');
      }
      this.fsm.transitionTo('LevelWon');
      // 胜利音效改由结算页打开时播放，与庆祝动画对齐
      this.events.emit({
        type: 'LevelWon',
        levelId: this.level.id,
        score: this.score,
      });
      this.deps.analytics.track('level_win', {
        levelId: this.level.id,
        score: this.score,
      });
      // 有剩余步数：先加成到彩色块再爆炸（开心消消乐），表现层播完消除后再 runMovesBonus
      if (this.movesLeft > 0 && this.board) {
        this.pendingMovesBonus = true;
        return;
      }
      this.finishAfterLevelWon();
      return;
    }
    if (this.board?.hasRevealedUnharvestedBuried()) {
      if (this.movesLeft <= 0) {
        return;
      }
      this.fsm.transitionTo('PlayerInput');
      this.ensurePlayableBoard();
      return;
    }
    if (this.movesLeft <= 0) {
```

```
      this.fsm.transitionTo('LevelFailed');
      this.playSfx('sfx_fail');
      this.events.emit({ type: 'LevelFailed', levelId: this.level.id });
      this.deps.analytics.track('level_fail', { levelId: this.level.id });
      void this.deps.ads.load('rewarded_revive');
      return;
    }
    this.fsm.transitionTo('PlayerInput');
    this.ensurePlayableBoard();
  }
  /**
   * 是否有待播放的「剩余步数加成」阶段。
   */
  public hasPendingMovesBonus(): boolean {
    return this.pendingMovesBonus;
  }
  /**
   * 执行剩余步数加成：步数打到彩色块变成闪光，再连锁爆炸加分。
   * 表现层应在胜利消除动画播完后调用。
   */
  public runMovesBonus(): boolean {
    if (!this.pendingMovesBonus || !this.board || !this.level) {
      return false;
    }
    if (this.fsm.getCurrent() !== 'LevelWon') {
      return false;
    }
    this.pendingMovesBonus = false;
    if (!this.fsm.transitionTo('MovesBonus')) {
      this.finishAfterLevelWon();
      return false;
    }
    const movesToConvert = this.movesLeft;
    const targets = this.pickMovesBonusTargets(movesToConvert);
    this.movesLeft = 0;
    for (const t of targets) {
      if (isBasicTile(this.board.getTile(t.row, t.col))) {
        this.board.setSparkle(t.row, t.col, true);
      }
    }
    this.events.emit({
      type: 'MovesBonusApplied',
      movesConverted: movesToConvert,
      targets: targets.slice(),
    });
    this.events.emit({ type: 'BoardChanged', version: this.board.version });
    const clear = new Set<number>();
    for (let i = 0; i < this.board.length; i += 1) {
      if (this.board.sparkles[i] === 1) {
        clear.add(i);
```

```
      }
    }
    expandSparkleBlasts(this.board, clear);
    const clearIndices = [...clear];
    if (clearIndices.length === 0) {
      this.events.emit({
        type: 'MovesBonusDone',
        bonusScore: 0,
        totalScore: this.score,
      });
      this.finishAfterLevelWon();
      return true;
    }
    const firstKinds: TileKind[] = clearIndices.map(
      (index) => this.board!.cells[index] as TileKind,
    );
    const playback = this.capturePlaybackSnapshot();
    const emptyMatches = this.finder.findMatches(this.board);
    const result = this.resolver.resolve(this.board, emptyMatches, {
      seed: this.level.seed + this.board.version + 911,
      initialClearIndices: clearIndices,
      spawnSpecials: true,
    });
    const bonusScore = result.cleared * 10;
    this.score += bonusScore;
    this.goals?.applyClearedKinds(result.clearedKinds);
    this.goals?.syncScore(this.score);
    this.emitGoalProgress();
    this.events.emit({
      type: 'TilesCleared',
      indices: clearIndices,
      kinds: firstKinds,
    });
    this.events.emit({
      type: 'ResolvePlayback',
      ...playback,
      waves: result.waves,
    });
    this.events.emit({ type: 'CascadeDone', depth: result.cascadeDepth });
    this.events.emit({ type: 'BoardChanged', version: this.board.version });
    this.events.emit({
      type: 'MovesBonusDone',
      bonusScore,
      totalScore: this.score,
    });
    this.finishAfterLevelWon();
    return true;
  }
  /**
   * 通关后进入粉碎或结算。
```

```
   */
  private finishAfterLevelWon(): void {
    if (!this.level) {
      this.fsm.forceTo('Settle');
      return;
    }
    if (this.level.crushEnabled) {
      this.beginCrushReward();
      return;
    }
    this.commitLevelCleared();
    this.lastSettleWon = true;
    if (
      this.fsm.getCurrent() === 'MovesBonus' ||
      this.fsm.getCurrent() === 'LevelWon'
    ) {
      this.fsm.transitionTo('Settle');
    } else if (this.fsm.getCurrent() !== 'Settle') {
      this.fsm.forceTo('Settle');
    }
  }
  /**
   * 按剩余步数随机挑彩色块（优先未闪光）。
   */
  private pickMovesBonusTargets(
    count: number,
  ): Array<{ row: number; col: number }> {
    if (!this.board || !this.level || count <= 0) {
      return [];
    }
    const random = createSeededRandom(this.level.seed + this.board.version + 4242);
    const candidates: Array<{ row: number; col: number; sparkle: boolean }> = [];
    for (let r = 0; r < this.board.size.rows; r += 1) {
      for (let c = 0; c < this.board.size.cols; c += 1) {
        const kind = this.board.getTile(r, c);
        if (!isBasicTile(kind)) {
          continue;
        }
        candidates.push({ row: r, col: c, sparkle: this.board.isSparkle(r, c) });
      }
    }
    if (candidates.length === 0) {
      return [];
    }
    candidates.sort((a, b) => Number(a.sparkle) - Number(b.sparkle));
    const plain = candidates.filter((c) => !c.sparkle);
    const pool = plain.length > 0 ? plain : candidates;
    const picks: Array<{ row: number; col: number }> = [];
    const used = new Set<number>();
    const n = Math.min(count, pool.length);
```

```
    let guard = 0;
    while (picks.length < n && guard < n * 8) {
      guard += 1;
      const idx = Math.floor(random() * pool.length);
      const cell = pool[idx]!;
      const key = this.board.index(cell.row, cell.col);
      if (used.has(key)) {
        continue;
      }
      used.add(key);
      picks.push({ row: cell.row, col: cell.col });
    }
    return picks;
  }
  /**
   * 先清掉盘面上已有的三连/四连（含冰封格），再在死局时重新排列。
   */
  private ensurePlayableBoard(): void {
    this.stabilizeBoard({ animate: true });
  }
  /**
   * 铺上棉花后只能洗牌，不能再结算三消，否则会提前削掉邻格棉花。
   */
  private shuffleAroundCloudLocks(seed: number): void {
    if (!this.board) {
      return;
    }
    for (let attempt = 0; attempt < 16; attempt += 1) {
      const noMatch = this.finder.findMatches(this.board).isEmpty();
      const hasMove = hasAnyValidMove(this.board, this.validator);
      if (noMatch && hasMove) {
        return;
      }
      shuffleUntilPlayable(this.board, seed + 199 + attempt * 17, 48, true);
    }
  }
  /**
   * 结算当前盘面已形成的匹配，必要时洗牌直到可走。
   * 洗牌只保证有合法交换，不会清掉已连成的 3/4，所以洗完必须再结算。
   */
  private stabilizeBoard(options: { animate: boolean }): void {
    if (!this.board || !this.level) {
      return;
    }
    for (let attempt = 0; attempt < 12; attempt += 1) {
      if (this.commitStandingMatches(options.animate)) {
        if (options.animate) {
          this.evaluateLevelOutcome();
          return;
        }
```

```
        continue;
      }
      if (hasAnyValidMove(this.board, this.validator)) {
        return;
      }
      const ok = shuffleUntilPlayable(
        this.board,
        this.level.seed + this.board.version + this.movesLeft * 17 + attempt * 31,
        48,
        true,
      );
      if (ok && options.animate) {
        this.events.emit({ type: 'BoardShuffled', reason: 'deadlock' });
        this.events.emit({ type: 'BoardChanged', version: this.board.version });
      }
    }
  }
  /**
   * 若盘面已有 ≥3 连（冰封格同样计入），立刻连锁消除，不必先滑动。
   * @returns 是否发生了消除
   */
  private commitStandingMatches(animate: boolean): boolean {
    if (!this.board || !this.level || !this.goals) {
      return false;
    }
    const matches = this.finder.findMatches(this.board);
    if (matches.isEmpty()) {
      return false;
    }
    if (animate && this.fsm.getCurrent() === 'PlayerInput') {
      this.fsm.transitionTo('Resolving');
    }
    const standingIndices = Array.from(matches.indices.subarray(0, matches.size));
    const firstKinds: TileKind[] = standingIndices.map(
      (index) => this.board!.cells[index] as TileKind,
    );
    const playback = this.capturePlaybackSnapshot();
    const result = this.resolver.resolve(this.board, matches, {
      seed: this.level.seed + this.board.version + 53,
    });
    if (result.cleared <= 0) {
      if (animate && this.fsm.getCurrent() === 'Resolving') {
        this.fsm.transitionTo('PlayerInput');
      }
      return false;
    }
    this.score += result.cleared * 10;
    this.goals.applyClearedKinds(result.clearedKinds);
    this.goals.syncScore(this.score);
    this.emitGoalProgress();
```

```
    if (animate) {
      this.playMatchPraiseSfx(Math.max(30, result.cleared * 10));
      this.events.emit({
        type: 'TilesCleared',
        indices: standingIndices,
        kinds: firstKinds,
      });
      this.events.emit({
        type: 'ResolvePlayback',
        ...playback,
        waves: result.waves,
      });
      if (result.fell.length > 0) {
        this.events.emit({
          type: 'TilesFell',
          moves: result.fell.map((f) => ({
            tileId: f.tileId,
            fromRow: f.fromRow,
            toRow: f.toRow,
            col: f.col,
          })),
        });
      }
      if (result.spawned.length > 0) {
        this.events.emit({
          type: 'TilesSpawned',
          indices: result.spawned.map((s) => this.board!.index(s.row, s.col)),
          tileIds: result.spawned.map((s) => s.tileId),
          kinds: result.spawned.map((s) => s.kind),
        });
      }
      this.events.emit({ type: 'CascadeDone', depth: result.cascadeDepth });
    }
    this.events.emit({ type: 'BoardChanged', version: this.board.version });
    return true;
  }
  /** 盘面是否已无任何小动物（洞格不算占用）。 */
  private isBoardFullyCleared(): boolean {
    if (!this.board) {
      return true;
    }
    for (let i = 0; i < this.board.length; i += 1) {
      const kind = this.board.cells[i]!;
      if (!isHole(kind) && kind !== TileKind.Empty) {
        return false;
      }
    }
    return true;
  }
  private capturePlaybackSnapshot(): {
```

```
    rows: number;
    cols: number;
    cells: Int8Array;
    tileIds: Int32Array;
    sparkles: Uint8Array;
    cloud: Uint8Array;
    ice: Uint8Array;
    egg: Uint8Array;
    vine: Uint8Array;
  } {
    const board = this.board!;
    return {
      rows: board.size.rows,
      cols: board.size.cols,
      cells: Int8Array.from(board.cells),
      tileIds: Int32Array.from(board.tileIds),
      sparkles: Uint8Array.from(board.sparkles),
      cloud: Uint8Array.from(board.cloud),
      ice: Uint8Array.from(board.ice),
      egg: Uint8Array.from(board.egg),
      vine: Uint8Array.from(board.vine),
    };
  }
  private emitGoalProgress(): void {
    if (!this.goals) {
      return;
    }
    if (this.board && this.iceAtStart > 0) {
      this.goals.syncClearIce(this.iceAtStart - this.board.countIce());
    }
    if (this.board && this.cloudsAtStart > 0) {
      this.goals.syncClearCloud(this.cloudsAtStart - this.board.countCloud());
    }
    if (this.board && this.gemsAtStart > 0) {
      this.goals.syncCollectGems(this.gemsAtStart - this.board.countGems());
    }
    if (this.board && this.eggsAtStart > 0) {
      this.goals.syncCollectChicks(this.eggsAtStart - this.board.countEggs());
    }
    if (this.board && this.vinesAtStart > 0) {
      this.goals.syncClearVines(this.vinesAtStart - this.board.countVines());
    }
    if (this.board) {
      const quakes = this.board.lastSnowmanQuakeCenters;
      if (quakes.length > 0) {
        this.board.lastSnowmanQuakeCenters = [];
        this.events.emit({ type: 'SnowmanQuake', centers: quakes });
      }
      this.goals.syncCollectSnowmen(this.board.harvestedSnowmen);
      this.goals.syncCollectPenguins(this.board.harvestedPenguins);
```

```
      for (const group of this.board.listBuriedGroups()) {
        if (!group.revealed || this.buriedNotified.has(group.id)) {
          continue;
        }
        this.buriedNotified.add(group.id);
        this.events.emit({
          type: 'BuriedRevealed',
          id: group.id,
          kind: group.kind,
          cells: group.cells.slice(),
        });
      }
    }
    this.events.emit({
      type: 'GoalProgress',
      progress: this.goals.getProgress().slice(),
      completed: this.goals.isAllCompleted(),
    });
  }
  /** 整只露出并展示完毕后收走一只企鹅/雪人。 */
  public collectBuriedGroup(groupId: number): boolean {
    if (!this.board || !this.goals) {
      return false;
    }
    const harvested = this.board.harvestBuriedGroup(groupId);
    if (harvested.length === 0) {
      return false;
    }
    this.buriedNotified.delete(groupId);
    this.goals.syncCollectSnowmen(this.board.harvestedSnowmen);
    this.goals.syncCollectPenguins(this.board.harvestedPenguins);
    this.events.emit({ type: 'BuriedHarvested', indices: harvested });
    this.events.emit({
      type: 'GoalProgress',
      progress: this.goals.getProgress().slice(),
      completed: this.goals.isAllCompleted(),
    });
    this.evaluateLevelOutcome();
    return true;
  }
  /**
   * 进入粉碎奖励环节。
   */
  private beginCrushReward(): void {
    if (!this.level) {
      this.fsm.transitionTo('Settle');
      return;
    }
    const duration = Math.max(0, this.level.crushDurationMs);
    this.crush = new CrushRewardModel(duration, this.crushTapPower);
```

```
    this.fsm.transitionTo('CrushReward');
    this.deps.analytics.track('crush_start', {
      levelId: this.level.id,
      durationMs: duration,
    });
    void this.deps.ads.load('rewarded_crush_extend');
  }
  private findLevelConfig(levelId: number): LevelConfig {
    return resolveMatch3LevelConfig(
      levelId,
      this.levelTable,
      this.getInviteCode() || 'local',
    );
  }
}
// 文件：src/services/MahjongHallService.ts
import type { IAnalytics, IStorage } from '../core';
import { StorageKeys } from '../core/ports/IStorage';
import {
  MAHJONG_BASE_SLOTS,
  MAHJONG_INFINITE_LEVEL_CAP,
  MAHJONG_REPLAY_COIN_COST,
  type MahjongToolId,
} from '../logic/mahjong/MahjongConfig';
import { getMahjongTheme } from '../logic/mahjong/MahjongThemeCatalog';
import { MahjongGame } from '../logic/mahjong/MahjongGame';
import { generateMahjongLevel, verifySolvablePile } from '../logic/mahjong/MahjongGenerator';
import {
  getMahjongOpsVersion,
  mahjongCoinCost,
  mahjongTimeoutMsForLevel,
  shouldMahjongOpsSampleLevel,
} from '../logic/mahjong/MahjongOpsConfig';
import {
  emptyMahjongProgress,
  normalizeMahjongProgress,
  recordMahjongClear,
  type MahjongHallProgressData,
} from '../logic/mahjong/MahjongProgress';
export type MahjongLadderCoinWallet = {
  getAvailable: () => number;
  spend: (amount: number) => boolean;
};
export class MahjongHallService {
  private data: MahjongHallProgressData = emptyMahjongProgress();
  private active: MahjongGame | null = null;
  private playerKey = 'local';
  private cloudSyncHook: (() => void) | null = null;
  private ladderCoinWallet: MahjongLadderCoinWallet | null = null;
  public constructor(
```

```
    private readonly storage: IStorage,
    private readonly analytics: IAnalytics,
  ) {}
  /** 本地麻将进度变更后触发云存档推送（由 GameSession 注入）。 */
  public setCloudSyncHook(hook: () => void): void {
    this.cloudSyncHook = hook;
  }
  /** 萌宠麻将馆消耗统一走梯子关卡折算金币。 */
  public setLadderCoinWallet(wallet: MahjongLadderCoinWallet): void {
    this.ladderCoinWallet = wallet;
  }
  public getSpendableCoins(): number {
    if (this.ladderCoinWallet) {
      return this.ladderCoinWallet.getAvailable();
    }
    return this.data.coins;
  }
  public setPlayerKey(key: string): void {
    this.playerKey = key.length > 0 ? key : 'local';
  }
  public async load(): Promise<void> {
    const saved = await this.storage.get<MahjongHallProgressData>(StorageKeys.MahjongHallProgress);
    this.data = normalizeMahjongProgress(saved);
  }
  public async persist(): Promise<void> {
    await this.storage.set(StorageKeys.MahjongHallProgress, this.data);
    this.cloudSyncHook?.();
  }
  public getProgress(): Readonly<MahjongHallProgressData> {
    return this.data;
  }
  public getCloudSlice(): MahjongHallProgressData {
    return normalizeMahjongProgress(this.data);
  }
  public loadProgress(data: MahjongHallProgressData): void {
    this.data = normalizeMahjongProgress(data);
    if (this.active && this.data.savedRun?.phase === 'playing') {
      if (this.active.level !== this.data.savedRun.level) {
        this.active = null;
      }
    } else if (this.active?.phase === 'playing') {
      this.active = null;
    }
  }
  public getActiveGame(): MahjongGame | null {
    return this.active;
  }
  public isLevelUnlocked(level: number): boolean {
    return level >= 1 && level <= this.data.highestUnlocked;
  }
```

```
  public bestStars(level: number): number {
    return this.data.starsByLevel[String(level)] ?? 0;
  }
  /** 入口直进对局：有未完成局则恢复，否则当前进度关（无限叠加的最高可挑战关）。 */
  public resolveEntryLevel(): number {
    if (
      this.data.savedRun?.phase === 'playing' ||
      this.data.savedRun?.phase === 'lost'
    ) {
      return this.data.savedRun.level;
    }
    if (this.data.blockedReplayLevel != null) {
      return this.data.blockedReplayLevel;
    }
    return this.data.highestUnlocked;
  }
  /** 无未完成局且本关已失败锁定时，不可免费再进。 */
  public canFreeEnterMahjongLevel(level: number): boolean {
    const safeLevel = Math.max(1, Math.floor(level));
    if (
      this.data.savedRun &&
      this.data.savedRun.level === safeLevel &&
      (this.data.savedRun.phase === 'playing' || this.data.savedRun.phase === 'lost')
    ) {
      return true;
    }
    return this.data.blockedReplayLevel !== safeLevel;
  }
  public enterFromHall(): MahjongGame {
    const level = this.resolveEntryLevel();
    if (!this.canFreeEnterMahjongLevel(level)) {
      throw new Error('MAHJONG_REPLAY_REQUIRED');
    }
    const resume =
      this.data.savedRun != null &&
      this.data.savedRun.level === level &&
      (this.data.savedRun.phase === 'playing' || this.data.savedRun.phase === 'lost');
    return this.startLevel(level, resume);
  }
  /** 本关重开：清存档并重新生成牌局（需已付金币或从失败 UI 触发）。 */
  public async restartLevel(level: number): Promise<MahjongGame> {
    if (this.active) {
      this.data.tools = { ...this.active.tools };
      this.active = null;
    }
    this.data.savedRun = null;
    this.data.blockedReplayLevel = null;
    await this.persist();
    return this.startLevel(level, false);
  }
```

```
  public startLevel(level: number, resume = true): MahjongGame {
    const safeLevel = Math.min(
      MAHJONG_INFINITE_LEVEL_CAP,
      Math.max(1, Math.floor(level)),
    );
    if (
      resume &&
      this.data.savedRun &&
      this.data.savedRun.level === safeLevel &&
      (this.data.savedRun.phase === 'playing' || this.data.savedRun.phase === 'lost')
    ) {
      if (
        this.data.savedRun.phase === 'playing' &&
        this.data.blockedReplayLevel === safeLevel
      ) {
        resume = false;
      } else {
        this.active = MahjongGame.fromSnapshot(this.data.savedRun);
        this.analytics.track('mahjong_level_resume', {
          level: safeLevel,
          phase: this.active.phase,
        });
        return this.active;
      }
    }
    if (!resume && this.data.blockedReplayLevel === safeLevel) {
      throw new Error('MAHJONG_REPLAY_REQUIRED');
    }
    const generated = generateMahjongLevel(safeLevel, this.playerKey);
    if (shouldMahjongOpsSampleLevel(safeLevel)) {
      const solvable = verifySolvablePile(generated.pile, MAHJONG_BASE_SLOTS);
      this.analytics.track('mahjong_ops_level_sample', {
        level: safeLevel,
        solvable,
        seed: generated.seed,
        typeCount: generated.typeCount,
        opsVersion: getMahjongOpsVersion(),
      });
    }
    const themeId = generated.themeId;
    const themeTileCount =
      getMahjongTheme(themeId)?.tiles.length ?? generated.typeCount;
    this.active = new MahjongGame(
      generated,
      themeId,
      { ...this.data.tools },
      this.data.guideSeen,
      mahjongTimeoutMsForLevel(safeLevel),
      themeTileCount,
    );
```

```
    if (safeLevel === 1 && !this.data.guideSeen) {
      this.active.guideShown = false;
    }
    this.data.savedRun = this.active.toSnapshot();
    void this.persist();
    const themeName = getMahjongTheme(themeId)?.name ?? themeId;
    this.analytics.track('mahjong_level_start', { level: safeLevel, themeId, themeName });
    this.analytics.track('mahjong_theme_exposure', { level: safeLevel, themeId, themeName });
    return this.active;
  }
  public trackTileTap(level: number): void {
    this.analytics.track('mahjong_tile_tap', { level });
  }
  public trackTripleClear(level: number, stars: number): void {
    this.analytics.track('mahjong_triple_clear', { level, stars });
  }
  public trackToolUse(tool: MahjongToolId, level: number): void {
    this.analytics.track('mahjong_tool_use', { tool, level });
  }
  public trackHallEnter(level: number): void {
    this.analytics.track('mahjong_hall_enter', { level });
  }
  public trackAutoNext(level: number): void {
    this.analytics.track('mahjong_auto_next', { level });
  }
  public trackThemeCdnFallback(requestedThemeId: string): void {
    this.analytics.track('mahjong_theme_cdn_fallback', { requestedThemeId });
  }
  public async saveActiveRun(): Promise<void> {
    if (!this.active || this.active.phase !== 'playing') {
      return;
    }
    this.data.savedRun = this.active.toSnapshot();
    await this.persist();
    this.analytics.track('mahjong_progress_save');
  }
  /** 失败瞬间写入 lost 快照，避免磁盘仍留 playing 可白嫖续玩。 */
  public async persistFailedRun(): Promise<void> {
    if (!this.active || this.active.phase !== 'lost') {
      return;
    }
    this.data.savedRun = this.active.toSnapshot();
    this.data.blockedReplayLevel = this.active.level;
    await this.persist();
  }
  public async clearSavedRun(): Promise<void> {
    this.data.savedRun = null;
    this.data.blockedReplayLevel = null;
    await this.persist();
  }
```

```
  public async finishActive(won: boolean): Promise<void> {
    if (!this.active) {
      return;
    }
    const game = this.active;
    if (won) {
      this.data = recordMahjongClear(this.data, game.level, game.starsThisRun, 0);
      this.data.tools = { ...game.tools };
      if (game.level === 1) {
        this.data.guideSeen = true;
      }
      this.analytics.track('mahjong_level_win', {
        level: game.level,
        stars: game.starsThisRun,
        coins: game.coinsThisRun,
      });
    } else {
      this.data.tools = { ...game.tools };
      this.data.savedRun = null;
      if (this.data.blockedReplayLevel == null) {
        this.data.blockedReplayLevel = game.level;
      }
      this.analytics.track('mahjong_level_fail', {
        level: game.level,
        reason: game.failReason ?? 'unknown',
      });
    }
    this.active = null;
    await this.persist();
  }
  public leaveTable(): void {
    if (this.active?.phase === 'playing') {
      this.data.savedRun = this.active.toSnapshot();
      void this.persist();
    } else if (this.active?.phase === 'lost') {
      void this.persistFailedRun();
    }
    this.active = null;
  }
  public spendCoins(amount: number): boolean {
    const cost = Math.floor(amount);
    if (cost <= 0) {
      return true;
    }
    if (this.ladderCoinWallet) {
      return this.ladderCoinWallet.spend(cost);
    }
    if (this.data.coins < cost) {
      return false;
    }
```

```
    this.data.coins -= cost;
    void this.persist();
    return true;
  }
  public buyTool(tool: MahjongToolId): boolean {
    if (!this.spendCoins(mahjongCoinCost())) {
      return false;
    }
    this.grantTool(tool, 'coin');
    return true;
  }
  /** 看广告等途径发放 1 个道具 */
  public grantTool(tool: MahjongToolId, source: 'coin' | 'ad' = 'ad'): void {
    this.data.tools[tool] += 1;
    if (this.active) {
      this.active.tools[tool] += 1;
    }
    void this.persist();
    if (source === 'coin') {
      this.analytics.track('mahjong_coin_buy_tool', { tool });
    }
  }
  public payReplay(coinCost = MAHJONG_REPLAY_COIN_COST): boolean {
    if (!this.spendCoins(coinCost)) {
      return false;
    }
    this.analytics.track('mahjong_coin_replay');
    return true;
  }
  public async unlockAdSlot(): Promise<boolean> {
    if (!this.active || this.active.adSlotUnlocked) {
      return false;
    }
    this.active.unlockAdSlot();
    this.data.savedRun = this.active.toSnapshot();
    await this.persist();
    this.analytics.track('mahjong_ad_slot');
    return true;
  }
  public async reviveAfterSlotFullAd(): Promise<boolean> {
    if (!this.active?.reviveMoveOutFromSlots()) {
      return false;
    }
    this.data.blockedReplayLevel = null;
    this.data.savedRun = this.active.toSnapshot();
    await this.persist();
    this.analytics.track('mahjong_slot_full_revive', { level: this.active.level });
    return true;
  }
}
```